

講義 25：Data Dictionary 總覽與 Global Type (授課順序：接在講義 9 (ALV) 之後、講義 21 之前)

對應練習：ex25 | 答案物件：Domain/DE ZTR25_SURPCT、DE ZTR25_ACTIVE (重用標準 Domain XFELD)、表 ZTR25_SURCHG (SM30)、Table Type ZTR25_TT_SURCHG + 程式 ZR_TR25_DDIC

本講重點

- Data Dictionary (SE11) 在系統裡的角色：一張物件地圖，程式與畫面共用同一份定義
- **Global Type** 回顧 (觀念已在講義 6 第 1.1 節講過)：重點放在動手建自己的 Global Type
- 為什麼要自建 Z 表：業務需求 + SM30 讓非工程師也能維護資料
- Check Table / 外鍵 / Search Help：概念總覽 (詳細動手在講義 21)
- 程式裡引用表格 / 結構的三種寫法，以及「該重用標準型別、還是自建」的判斷
- **DDIC Table Type**：把講義 4 的區域表格型別升級成全域可共用的 SE11 物件
- 完整案例：SM30 維護的「航空公司旺季加成」設定表，重用標準 Data Element 免費拿到 Check Table 與 Search Help

1. Data Dictionary 是什麼：一張系統地圖

從講義 6 起你一直在用 DDIC (TYPE scarr-carrid)，但還沒看過它的全貌。SE11 管理的物件其實有一整組層級，各司其職：

物件	管什麼	誰在用
Domain (值域)	技術屬性：型別、長度、小數位、值域清單	Data Element
Data Element (資料元素)	語意：標籤、F1 說明、Search Help 掛勾 (講義 21 三層件)	表格欄位、程式變數
Structure (結構)	純欄位組合，不對應資料庫表 (如畫面用的暫存結構)	程式、畫面
Table (透明表)	對應資料庫的真實表	Open SQL、SM30、程式
Table Type	表格型別 (「很多列」的定義)，可跨程式共用	方法/FM 的表格參數
Search Help	F4 選單來源	畫面欄位、Data Element
Lock Object	產生 ENQUEUE/DEQUEUE FM，防止多人同時改同一筆 (講義 21 提過，進階課題)	程式

關鍵觀念：這些定義只寫一次，程式 (用 TYPE) 跟畫面 (Dynpro/SM30) 共用同一份——這就是接下來 Global Type 觀念的基礎。

2. Global Type 回顧：為什麼要引用 DDIC 型別，不要寫死

Global Type 的觀念 (內建型別 / Local Type / Global Type 三層、判斷準則、為什麼不寫死長度) 已經在講義 6 第 1.1 節講過，這裡不重複，只回顧最關鍵的一點，接著把重心放在「怎麼動手建自己的 Global Type」。

2.1 回顧：寫死長度是最陰險的 bug

物料號碼 MATNR 從 18 碼加長到 40 碼：引用 Global Type 的程式 (TYPE mara-matnr) 升級後一行都不用改；寫死 TYPE c LENGTH 18 的程式資料被默默截斷，不當機、不報錯，可能幾個月後才被發現。型別的定義權交給系統唯一的來源 (Domain / Data Element)，程式只負責「引用」——本講後面的每一種做法，都是在這個前提下，決定「引用現成的」還是「自己建一個」。

2.2 什麼時候該重用標準型別、什麼時候該自建

情境	做法
語意跟標準欄位完全一樣 (航空公司代碼、物料號碼、客戶編號...)	重用標準 Data Element (TYPE scarr-carrid 或直接 TYPE s_carr_id)
你們公司獨有的業務概念，標準系統沒有對應語意 (自訂的加成比例、自訂的審核狀態...)	自建 Domain/Data Element (講義 21 的三層件流程)

判斷原則很簡單：先搜尋標準有沒有語意相符的 Data Element，有就重用；沒有才自建——本講最後的案例會示範重用標準型別能省下多少工。

3. 為什麼要自建 Z 表：業務需求 + SM30 開放維護

標準系統管不到的東西，例如客製化的參數設定、控制開關、公司自己的業務對照表，就需要自建 Z 表。核心價值不是「能存資料」(那用什麼工具都能存)，而是：

- **SM30 讓不會寫程式的人也能維護資料**：業務人員自己在 SM30 改參數、加一筆設定，不需要工程師介入、不需要走傳輸請求 (資料本身不需要 TR，只有表結構變更才需要)。
- 跟講義 21 的分工：講義 21 教你「怎麼建」(Domain/DE/Table/SM30/外鍵/Search Help 手把手流程)；本講先講「為什麼建、什麼時候該重用 vs 自建」，兩講合起來才是完整的 DDIC 觀念。

4. Check Table / 外鍵 / Search Help：總覽先看懂

講義 21 會帶你完整動手建一次 (兩張都是自建的 Z 表)；這裡先看懂三個名詞在解決什麼問題，本講最後的案例則刻意示範另一種更常見的組合：

名詞	解決什麼問題
Check Table (檢查表)	誰是「合法值清單」——可以是自建的 Z 表，也可以直接是標準表 (如 SCARR)
Foreign Key (外鍵)	讓畫面輸入時擋掉不在合法清單裡的值 (只管畫面，不管 Open SQL——講義 21 會實測驗證)
Search Help (搜尋輔助)	給 F4 選單；如果欄位重用了已經掛好 Search Help 的標準 Data Element，你完全不用自己建

本講案例：Check Table 直接指向標準表 **SCARR**，欄位重用標準 Data Element **S_CARR_ID**——順便驗證它已經掛好的 Search Help 直接生效，一個都不用自己建，比講義 21（兩個都要自建）省事得多，也更貼近實務常態（大部分自建表的關聯欄位，另一端往往是標準主檔）。

5. 程式中引用 DDIC 物件的三種寫法

DATA gs_row TYPE ztable.	" 整列：跟表格結構同型別（work area）
DATA gv_val TYPE ztable-field.	" 單一欄位：跟表格該欄位同型別
DATA gt_rows TYPE STANDARD TABLE OF ztable.	" 整張表：多筆集合（講義 4 學過的表格型別）

欄位型別還有一種更直接的寫法——繞過表格路徑，直接引用 **Data Element**：

DATA gv_carrid TYPE s_carr_id.	" 直接引用 Data Element
DATA gv_carrid TYPE scarr-carrid.	" 透過表格路徑引用（兩者型別完全相同，因為 SCARR-CARRID 本來就是用 S_CARR_ID 定義的）

兩種寫法效果一樣，選哪個看語境：如果變數的意義跟某張特定表綁得很緊，用 **TYPE 表-欄位** 讀起來更清楚；如果是通用的「一個航空公司代碼」，直接 **TYPE s_carr_id** 更能表達「這是一個標準概念，不是某張表專屬的」。

6. 案例：SM30 維護的「航空公司旺季加成」設定表

情境：財務單位想針對特定航空公司的航班設定「旺季加成百分比」，希望自己在 SM30 維護、不用每次找工程師改程式；報表要能讀這張表，把加成反映進營收試算。

6.1 建立 Domain 與 Data Element（自建——因為「加成百分比」是我們公司獨有的概念）

- **Domain ZTR25_SURPCT**：Data Type **DEC**，Length **6**、Decimals **2**（DEC 的 Length 要把小數點也算進去，**100.00** 需要 6 碼）；Value Range 頁籤設 Interval **0~100**（DEC 型別的值域上下限只接受整數，填 **0.00 / 100.00** 會啟用失敗）→ 啟用
- **Data Element ZTR25_SURPCT**：參考上面的 Domain；Field Label 填「旺季加成百分比」→ 啟用

6.2 第三種 Global Type 模式：重用標準 Domain、自己補標籤

ACTIVE（是否啟用加成）圖方便的話很容易直接用內建型別 **CHAR 1** 打發——這是個陷阱：SE11 建表時該欄位沒有任何 Data Element 可以提供標籤，SM30 產生的維護畫面欄位標題會顯示成通用符號 **+**，而且完全沒有 F4 下拉選單（沒有 Domain 固定值清單可以參考）。

正確做法：SAP 標準已經有一個通用的「是 / 否」值域 **Domain XFELD**，內建 **X = 是**、空白 = 否兩個固定值——但它掛的標準 Data Element **XFELD** 本身故意不帶任何標籤（設計上就是留給每張表自己命名，因為「是 / 否」用在哪張表都語意不同）。所以我們建一個新 Data Element **ZTR25_ACTIVE**，**Domain** 選標準的 **XFELD**（不自建 **Domain**），只補上自己的標籤「加成啟用」：

層	做法
Domain	重用標準 XFELD （技術屬性 + 固定值清單，一個都不用自己建）
Data Element	自建 ZTR25_ACTIVE （只補標籤，型別/值域全部繼承自 XFELD ）

這是跟前面兩種模式都不同的**第三種 Global Type** 用法：

模式	例子	重用什麼	自建什麼
整個 Data Element 重用	CARRID 用 S_CARR_ID	型別 + 標籤 + Search Help 全部	什麼都不用建
完全自建	SURCHARGE_PCT 用 ZTR25_SURPCT	什麼都不重用	Domain + Data Element 都自己定義
重用 Domain、自建標籤	ACTIVE 用 ZTR25_ACTIVE (Domain 是 XFELD)	技術屬性 + 固定值清單	只補一個貼近自己業務語境的標籤

判斷原則：遇到「是 / 否」「啟用 / 停用」這類通用的旗標概念，先搜尋標準有沒有像 **XFELD** 這樣的通用 **Domain**——有的話重用它的技術屬性與固定值，自己只需要補標籤，比整組自建省事得多。

6.3 建立表 **ZTR25_SURCHG**

欄位	Key	型別來源	說明
MANDT	✓	Data Element MANDT	client
CARRID	✓	標準 Data Element S_CARR_ID (不是自建！)	航空公司代碼，同時是 Key 也是外鍵
ACTIVE		Data Element ZTR25_ACTIVE (6.2 自建，重用標準 Domain XFELD)	是否啟用加成
SURCHARGE_PCT		Data Element ZTR25_SURPCT	加成百分比 (自建)
UPDUSER		Data Element SYUNAME	異動者
UPDDATE		Data Element SYDATUM	異動日

CARRID 欄位的 **Foreign Key** 對話框：Check Table 填 **SCARR** (標準表，不是自建的 Z 表！)，Cardinality 選 **[0..1] : 1** (每家航空公司最多一筆加成設定，**0..1** 是因為不是每家都設定過)，Foreign Key Fields 讓系統自動帶出 **CARRID = CARRID**；打開 Screen Check。存檔啟用。

DDL 檢視長這樣 (跟講義 21 的外鍵語法同一套，只是 Check Table 換成標準表)：

```
key carrid : s_carr_id not null
  with foreign key [0..1,1] scarr
    where mandt = ztr25_surchg.mandt
      and carrid = ztr25_surchg.carrid;
```

6.4 建 SM30 維護畫面、驗證「免費拿到」的東西

- Utilities → Table Maintenance Generator：Authorization Group **&NC&**、Function Group **ZFG_TR25**、one step → 產生
- SM30 → **ZTR25_SURCHG** → 新增兩筆：**AA** / 啟用 / **15.00**、**LH** / 啟用 / **10.00**
- 驗證重用標準型別的三个免費好處：

1. **CARRID** 欄位按 **F4**：選單直接出現（航空公司清單）——因為 **S_CARR_ID** 早就掛好了標準 Search Help，我們什麼都沒建
2. 輸入一個 **SCARR** 沒有的代碼（如 **ZZ**）：畫面直接擋下來——外鍵在起作用，Check Table 是標準表一樣有效
3. **ACTIVE** 欄位有正確的欄位標題（不是通用符號 +），按 **F4** 出現「是 / 否」選單——重用 **XFELD** 的固定值清單換來的，一個 Domain 都沒自己建

對照講義 21：**ZTR21_STUD-KLASSE** 因為是全新自建概念，Search Help 要整個手工建；這裡因為重用了標準 Data Element，Search Help **完全不用建**——這就是 Global Type 省下的工。

6.5 程式讀取：Global Type 宣告 + 加成營收試算

```
DATA gs_surchg TYPE ztr25_surchg.      " 整列：Global Type 宣告
DATA gv_carriid TYPE s_carr_id.        " 直接引用標準 Data Element

TYPES: BEGIN OF ty_rev,
        carriid      TYPE s_carr_id,
        carrname     TYPE scarr-carrname,
        connid       TYPE sflight-connid,
        fldate       TYPE sflight-fldate,
        seatsocc     TYPE sflight-seatsocc,
        price        TYPE sflight-price,
        active       TYPE ztr25_surchg-active,
        surcharge_pct TYPE ztr25_surchg-surcharge_pct,
        revenue      TYPE p LENGTH 12 DECIMALS 2,
        revenue_adj  TYPE p LENGTH 12 DECIMALS 2,
      END OF ty_rev.

DATA gt_rev TYPE STANDARD TABLE OF ty_rev.
FIELD-SYMBOLS <ls_rev> TYPE ty_rev.      " 指向內表的一列（講義 16）

SELECT f~carriid c~carrname f~connid f~fldate f~seatsocc f~price
       s~active s~surcharge_pct
  INTO CORRESPONDING FIELDS OF TABLE gt_rev
 FROM sflight AS f
 INNER JOIN scarr AS c ON c~carriid = f~carriid
 LEFT OUTER JOIN ztr25_surchg AS s ON s~carriid = f~carriid " 沒設定過的公司也要出現，
用 LEFT OUTER
 WHERE f~seatsocc > 0
 ORDER BY f~carriid f~connid f~fldate.

LOOP AT gt_rev ASSIGNING <ls_rev>.
  <ls_rev>-revenue = <ls_rev>-price * <ls_rev>-seatsocc.
  IF <ls_rev>-active = 'X'.
    <ls_rev>-revenue_adj = <ls_rev>-revenue * ( 1 + <ls_rev>-surcharge_pct / 100 ).
  ELSE.
    <ls_rev>-revenue_adj = <ls_rev>-revenue.
```

```
ENDIF.  
ENDLOOP.
```

`LEFT OUTER JOIN` 是關鍵 (講義 11 學過) : 沒被財務設定過的航空公司, `active / surcharge_pct` 是初始值, `revenue_adj` 自然等於原始營收——不用另外寫 IF 判斷「有沒有設定」。

6.6 把「表格型別」也升級成 Global Type : 建立 DDIC Table Type

講義 4 教過 `TYPES tt_student TYPE STANDARD TABLE OF ty_student`——但那是 **Local Type**, 只有寫在那支程式裡看得到。講義 8 提過表格型別可以拿來宣告 FORM / FM 的參數 (`FORM show_list USING it_students TYPE tt_student.`), 但如果兩支不同程式都要用同一種表格當參數, 各自宣告一份 Local Type 只是把同一件事寫兩遍——改一個欄位, 兩邊都要記得改, 正是 Global Type 一開始要解決的問題 (第 2 節), 只是這次問題發生在「表格」這個層級, 不是單一欄位。

解法是 SE11 建一個全域的 **Table Type** (DDIC 物件型別 `TTYP`), 跟 Domain / Data Element 一樣只寫一次、全系統共用:

1. SE11 → Data Type → 輸入 `ZTR25_TT_SURCHG` → Create → 選 **Table Type**
2. **Line Type** (每一列的長相) : 直接填 `ZTR25_SURCHG`——表格本身也可以拿來當 **Line Type**, 不用重新定義欄位。這是另一層 Global Type 的好處: 以後 `ZTR25_SURCHG` 加欄位, `ZTR25_TT_SURCHG` 自動跟著多那個欄位, 兩邊零維護成本
3. **Access Mode** : 選 **Standard Table** (對照講義 4 : STANDARD / SORTED / HASHED 三選一), Key 用預設 (Default Key) 即可
4. 存檔、套件 `$TMP`、啟用

用起來跟本地表格型別語法一模一樣, 只是型別名稱換成 DDIC 物件:

```
DATA gt_surchg TYPE ztr25_tt_surchg.          " 引用全域 Table Type, 跟 TYPE STANDARD  
TABLE OF 效果相同  
  
FORM load_surchg_config CHANGING ct_surchg TYPE ztr25_tt_surchg.  
  SELECT * FROM ztr25_surchg INTO TABLE ct_surchg.  
ENDFORM.
```

`load_surchg_config` 這個 FORM 的簽名現在可以被任何程式照抄使用 (甚至改寫成 FM 的 TABLES / EXPORTING 參數, 講義 15 學過), 因為型別定義在 DDIC, 不是鎖死在某支程式裡——這就是「表格級別」的 Global Type。

判斷原則跟前面幾種模式一致: 只有本程式用就地 `TYPES` 宣告 (講義 4) 就好, 不用每個表都跑一次 SE11; 多支程式 / FM / 方法要共用同一種表格參數, 才值得升級成 DDIC Table Type。

7. 常見錯誤與陷阱

症狀	原因
SAP 升級後某欄位資料被截斷	程式宣告用了寫死的長度, 沒有引用 Global Type (2.1 的真實案例)

症狀	原因
自建 Data Element 卻發現標準早就有一個一模一樣的	建之前沒有先搜尋標準——語意重複的型別會讓維護更亂
外鍵 Check Table 選了自建 Z 表，其實應該指標準表	沒想清楚「合法值清單本來就已經存在」（如本例的 SCARR）
F4 選單自己手動建了一個 Search Help，其實不用	沒發現重用的 Data Element 早就掛好標準 Search Help
LEFT OUTER JOIN 忘記用，沒設定的資料整筆消失	誤用 INNER JOIN——回顧講義 11
SM30 資料改了以為要走傳輸請求	表結構才需要 TR；表資料維護不需要（除非 Delivery Class 特別設定）
SM30 欄位標題顯示通用符號 +、也沒有 F4 選單	欄位直接用內建型別（如 CHAR 1），沒有掛任何 Data Element——通用旗標欄位改用「重用標準 Domain（如 XFELD）+ 自建 Data Element 補標籤」（見 6.2）
每支程式都各自宣告一份幾乎一樣的 TYPES tt_xxx TYPE STANDARD TABLE OF ...	這種表格型別其實多支程式在共用，該升級成 DDIC Table Type（見 6.6），而不是各自維護一份 Local Type

8. 課堂練習

完成 ex25：建自訂 Domain/DE（加成百分比）+ 一張重用標準 Data Element 當 Key 又當外鍵的表、Check Table 指向標準 SCARR、SM30 維護畫面、一個引用該表當 Line Type 的 DDIC Table Type，再寫程式驗證「重用型別免費拿到 F4 與外鍵檢查」，並用 Global Type 宣告（含表格級的 Table Type）完成一份加成營收報表。